

beginner → advanced :

- **Duration:** 4 weeks
- **Schedule:** 3 days/week
- **Time:** 2 hours/day
- **Total time:** 24 hours

This plan balances **concepts (≈45%)**, **hands-on coding (≈45%)**, and **review/projects (≈10%)**.



Weekly Structure (Recommended)

Each 2-hour session:

- **30 min** – Concept explanation
- **60 min** – Guided coding + exercises
- **30 min** – Practice challenge / review



Week 1 – Python Fundamentals (Beginner)

Day 1: Getting Started with Python

Concepts

- What is Python & where it's used
- Installing Python & VS Code / Jupyter
- Running Python scripts
- Comments & syntax basics

Topics

- `print()`
- Variables
- Data types: `int` , `float` , `str` , `bool`
- Type conversion

Practice

- Simple calculator
- User input program

Day 2: Control Flow

Concepts

- Decision making
- Logical thinking

Topics

- `if` , `elif` , `else`
- Comparison & logical operators
- Indentation

Practice

- Grade calculator
- Odd/even checker
- Simple login simulation

Day 3: Loops

Concepts

- Iteration and repetition

Topics

- `for` loop
- `while` loop
- `range()`
- `break` , `continue`

Practice

- Number guessing game
- Multiplication table
- Pattern printing

Week 2 – Data Structures & Functions (Intermediate)

Day 4: Data Structures

Concepts

- Storing & organizing data

Topics

- Lists
- Tuples
- Sets
- Dictionaries

Practice

- Student records
- Shopping cart system
- Frequency counter

Day 5: Functions

Concepts

- Code reusability

Topics

- Defining functions
- Parameters & return values

- Default arguments
- Scope (local vs global)

Practice

- Calculator using functions
- Password validator
- Reusable utility functions

Day 6: Strings & File Handling

Concepts

- Working with text & files

Topics

- String methods
- String formatting (f-strings)
- File read/write (open , with)

Practice

- Text analyzer
- Log file reader
- Save/load user data

Week 3 – Object-Oriented & Advanced Concepts

Day 7: Object-Oriented Programming (OOP)

Concepts

- Modeling real-world objects

Topics

- Classes & objects
- `__init__`
- Methods & attributes
- `self`

Practice

- Bank account class
- Student management system

Day 8: Advanced OOP & Error Handling

Concepts

- Robust program design

Topics

- Inheritance
- Polymorphism
- Encapsulation
- Exceptions (`try/except`)

Practice

- Library system
- Custom error handling

Day 9: Modules & Virtual Environments

Concepts

- Code organization

Topics

- Importing modules

- Creating modules
- pip
- Virtual environments

Practice

- Build a utility module
- Use external libraries (`math` , `random`)

Week 4 – Advanced Python & Projects

Day 10: Advanced Python Features

Concepts

- Writing cleaner & faster code

Topics

- List/dict comprehensions
- Lambda functions
- `map()` , `filter()` , `reduce()`
- `zip()`

Practice

- Data transformation tasks

Day 11: Working with Libraries

Concepts

- Real-world Python usage

Topics (Choose based on goal)

- `requests` (APIs)

- json
- datetime
- Intro to pandas OR sqlite3

Practice

- API data fetcher
- Simple database app

Day 12: Final Project & Review

Project Options

- CLI To-Do App
- Expense Tracker
- Student Management System
- Mini API-based app

Activities

- Project planning
- Coding
- Debugging
- Best practices review

Outcomes After 4 Weeks

By the end, learners should be able to:

- Write clean Python programs
- Understand OOP principles
- Work with files, APIs, and libraries
- Build small real-world projects
- Be ready for **data science, web, or automation paths**



Optional Add-ons

- Daily mini challenges (15 min)
- Weekly quiz
- Git & GitHub basics
- LeetCode-style problems